Math Reasoning with Adaptive MCTS and Process-Level Scoring

Ruochen Wu (rw3152) Hammer Niu (hn2477) Shengbo Yi (sy3302)

May 5, 2026

Abstract

This project investigates whether search and process-level scoring can make LLM-based math reasoning more reliable than a single direct reasoning path. We implement a baseline Monte Carlo Tree Search (MCTS) solver, an adaptive search variant, and a PPM-compatible process scorer that combines a learned Process Preference Model with a deterministic verifier. The system is exposed through command-line scripts, a Streamlit comparison interface, and a FastAPI backend. Deterministic ablations show that adaptive search improves controlled answer selection from 0/3 to 3/3 while reducing model calls from 6.0 to 2.0 per problem, and that the hybrid scorer corrects 3/3 deliberately constructed process-score misrankings. The repository also includes scripts for MATH Level 5 and OlympiadBench evaluation when API keys are available.

1 Introduction and Problem Formulation

Large language models can solve many short math problems, but direct generation often commits to a single reasoning path. For multi-step problems, a small intermediate error can propagate into a fluent but incorrect final answer. This project studies a more deliberate alternative: search over multiple candidate reasoning steps, score the quality of intermediate process steps, and return the best trajectory.

The project goal is to reproduce a working math-reasoning baseline and improve it with two concrete innovations: an adaptive MCTS search policy and a process-level scorer that combines learned PPM scoring with a lightweight math verifier. The central research question is: given the same math problem and the same generator model, can structured search and process scoring improve which reasoning path is selected?

The system is packaged as a runnable command-line, Streamlit, and FastAPI demo. The final report, repository, and demo artifacts are intentionally aligned with the course rubric so that the code, presentation, and evaluation describe the same end-to-end system.

Repository: `https://github.com/HammerNiu/Math-Reasoning`

Demo artifact: `https://github.com/HammerNiu/Math-Reasoning/blob/main/docs/demo_video.gif`

2 Related Work and Background

The project is inspired by search-based reasoning systems such as rStar-Math, where candidate reasoning trajectories are generated, scored, and improved over multiple rounds. MCTS provides a natural framework for balancing exploration and exploitation: selection chooses promising nodes, expansion adds candidate reasoning steps, simulation estimates path quality, and backpropagation updates node values. Process preference models complement this by scoring the quality of intermediate steps rather than judging only the final answer.

This project also relates to verifier-guided generation. Final-answer verification is often too late: a path can be fluent and internally inconsistent before the answer line appears. Process supervision instead asks whether each intermediate step is useful, mathematically grounded, and likely to lead to a correct final state. Our implementation uses this idea in two forms: a neural PPM trained on preference pairs and a deterministic verifier that provides a fallback score when no trained checkpoint is available.

3 System Design and Methodology

The system has four main components. First, a model interface supports OpenAI, DeepSeek, Anthropic, and local Ollama models. Second, the MCTS controller generates and searches over candidate steps. Third, the process scorer ranks steps before expansion. Fourth, the demo layer presents baseline and improved systems side by side.

3.1 End-to-End Workflow

At inference time, the system receives a problem string and initializes it as the root state of a search tree. The generator model proposes candidate next steps using a prompt that asks for explicit `STEP:` lines and a `FINAL ANSWER:` marker when the problem is solved. MCTS then selects and expands nodes. For the improved system, candidate steps are de-duplicated, scored, and pruned before

expansion. After simulation, rewards are backpropagated through the tree and the best child is returned as the selected reasoning action.

The design keeps policy and reward responsibilities separate. The policy model generates candidate steps. The reward side can use a trained PPM, a separate critic model, the deterministic verifier, or a hybrid PPM-verifier scorer. This separation makes the system easier to inspect and cheaper to adapt: improving the scorer does not require changing the generator.

3.2 Search Innovation

The baseline MCTS preserves a simple search strategy. The adaptive strategy adds larger branching near the root, duplicate removal, diversity-aware scoring, weak-branch pruning, early stopping, evaluation caching, and context/token controls. These changes are implemented in `src/core/mcts.py`.

The adaptive strategy changes three decisions. First, it asks for more candidate steps near the root, where early commitment is most costly. Second, it uses a simple value-diversity score to avoid spending all simulations on near-duplicate branches. Third, it stops early when a high-confidence terminal answer is found. These changes directly target the baseline failure mode: following the first plausible but weak branch.

3.3 Scoring Innovation

The scoring-side innovation is a PPM-compatible verifier and hybrid scorer in `src/core/scoring.py`. The verifier rewards mathematical progress, explicit operations, verification language, and final-answer markers. It penalizes vague guesses and unrelated steps. When a trained PPM checkpoint is available, the hybrid scorer combines the learned PPM score with the verifier; when no checkpoint is available, the verifier still provides a deterministic process score.

The verifier is not intended to replace proof checking. Instead, it is a low-cost process prior. It increases scores for steps that contain mathematical operations, explicit solving verbs, or verification language, and decreases scores for guesses, vague language, or unrelated actions. The hybrid scorer normalizes raw PPM logits into a 0–1 range and interpolates them with the verifier score. This gives the project a scoring-side innovation that works both before and after PPM training.

3.4 Demo and API Design

The Streamlit interface is designed for the presentation rubric: it runs the same input through baseline MCTS and improved MCTS, then displays selected steps, reasoning trajectories, model-call counts, pruned branches, and latency. The FastAPI backend exposes the same behavior through `/solve`, `/compare-models`, and `/health`. This makes the system usable as both a live demo and a reproducible service endpoint.

4 Implementation and Data

The main implementation files are:

- `src/core/mcts.py`: MCTS, adaptive search, top-k process pruning.
- `src/core/ppm.py`: neural process preference model and trainer.
- `src/core/scoring.py`: verifier and hybrid process scorer.
- `app.py`: side-by-side comparison demo.
- `backend/main.py`: FastAPI endpoints for solving and comparison.
- `tools/`: trajectory collection, PPM training, deterministic ablations.
- `experiments/run_experiment.py` and `eval.py`: MATH and OlympiadBench evaluation.

Trajectory data can be generated by running `tools/collect_trajectories.py`. Preference pairs are then used to train the PPM with `tools/train_ppm.py`. The full benchmark path uses MATH Level 5 and OlympiadBench through HuggingFace `datasets`.

4.1 Preference Pair Construction

The PPM training pipeline converts trajectories into pairwise preferences. Steps from high-reward trajectories become preferred examples, while steps from low-reward trajectories become non-preferred examples. The trainer embeds each step using a local sentence-transformer model and optimizes a hinge-style ranking loss so preferred steps receive higher values than non-preferred steps. This design avoids fine-tuning the generator model and keeps training local.

4.2 Configuration and Reproducibility

Configuration lives in `config/default.json`. The repository includes both cloud and local model paths: OpenAI, DeepSeek, Anthropic, and Ollama. Deterministic ablation scripts require no API keys, while full benchmark runs require model credentials and dataset access. The README lists exact commands for compiling, running ablations, launching the demo, training PPM checkpoints, and running benchmark comparisons.

5 Evaluation and Results

The committed evaluation contains deterministic no-key ablations plus scripts for API-backed benchmark runs. The deterministic experiments are intentionally small but isolate the two required algorithmic changes without depending on cloud API randomness.

5.1 Search Ablation

The search ablation compares baseline MCTS against adaptive MCTS on three scripted math problems. The baseline is intentionally exposed to a distracting first candidate. Adaptive MCTS should identify the useful final step sooner.

Table 1: Member 1 deterministic search ablation.

Strategy	Accuracy	Nodes	Calls	Pruned
Baseline MCTS	0/3	3.0	6.0	0.0
Adaptive MCTS	3/3	1.0	2.0	2.0

Qualitatively, the baseline chooses the first distracting candidate in each controlled problem: an unchecked arithmetic or algebraic guess. Adaptive MCTS promotes the terminal or mathematically grounded candidate because its heuristics reward explicit solution progress and penalize vague guessing. This supports the Member 1 claim that search-side pruning and early stopping improve both quality and efficiency in controlled cases.

5.2 Scoring Ablation

The scoring ablation uses a deliberately weak scorer that overvalues guesses. The hybrid verifier scorer corrects all three mis-rankings by preferring mathematically useful steps.

Table 2: Member 2 deterministic scoring ablation.

Task	Old scorer fixed?	Hybrid fixed?
Linear equation	No	Yes
Addition	No	Yes
Derivative	No	Yes

The scoring ablation is intentionally adversarial: the old scorer is biased toward guesses. The hybrid scorer fixes this by applying the verifier prior. For example, in the linear-equation case, the weak scorer prefers "Guess $x = 4$ without checking the equation," while the hybrid scorer prefers "Subtract 3 from both sides to get $x = 2$." This demonstrates the task-split requirement for error-analysis examples showing corrected mis-ranking cases.

5.3 End-to-End Evaluation

The Streamlit demo runs the same input problem through baseline MCTS and improved MCTS with process scoring. It reports selected step, trajectory, model calls, pruned branches, and latency. The full MATH/OlympiadBench protocol is implemented in `experiments/run_experiment.py` and `eval.py`.

The full benchmark protocol compares three configurations on held-out MATH Level 5 problems: baseline MCTS, adaptive MCTS, and adaptive MCTS with PPM top-k pruning. Metrics include accuracy, average model calls, latency, and PPM pruned branches. A separate evaluation script supports direct prompting, MCTS-only, and MCTS+PPM on MATH or OlympiadBench. These scripts are included so the final API-backed results can be reproduced exactly from the repository.

5.4 Failure Analysis

The main remaining risks are scorer noise and benchmark variance. A weak PPM can prune useful branches if used without enough training data; therefore the system supports verifier-only scoring and documents that top-k PPM pruning should be enabled only after trajectory collection and training. Answer checking for competition math is also imperfect because equivalent expressions can appear in different formats. The evaluation scripts use boxed-answer extraction, LaTeX normalization, numeric comparison, and token matching, but manual review remains useful for borderline cases.

6 Discussion

The controlled ablations show that the two algorithmic modifications work as intended: adaptive search reduces wasted exploration and the hybrid scorer corrects process-level mis-rankings. The main limitation is that deterministic ablations are smaller than full benchmark evaluation. The repository therefore includes the complete benchmark scripts and instructions so the final API-backed run can be reproduced before submission.

6.1 Division of Work

The implementation matches the three-person task split. Member 1 owns the MCTS search modification and ablation. Member 2 owns the PPM/verifier scoring modification and mis-ranking analysis. Member 3 owns the comparison demo, repository organization, and evaluation harness. All three components are represented in code, documentation, and presentation materials rather than being separated into only coding or writing tasks.

6.2 Submission Readiness

The repository now contains the report source, rubric checklist, demo guide, experiment analysis, deterministic result tables, and a demo GIF artifact. For CourseWorks, the final step is to export this report as `rw3152_hn2477_sy3302.pdf`, confirm the GitHub and demo links are accessible, and optionally replace the GIF with a narrated hosted video.

Future work includes training a larger PPM on more trajectories, adding richer mathematical verification, and evaluating on broader reasoning tasks beyond math.

7 Conclusion

This project delivers a coherent math-reasoning system with a baseline, two implemented innovations, runnable demos, deterministic evidence, and reproducible benchmark scripts. The implementation aligns the report, demo, and repository around a single end-to-end workflow: generate candidate reasoning steps, search over them, score process quality, and return the strongest trajectory.

References

[1] Xinyu Li et al. 2024. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking.

[2] Levente Kocsis and Csaba Szepesvari. 2006. Bandit Based Monte-Carlo Planning.

[3] Dan Hendrycks et al. 2021. Measuring Mathematical Problem Solving With the MATH Dataset.

[4] OlympiadBench. A benchmark for olympiad-level bilingual multimodal scientific reasoning.